

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Vanessa Paola Laje Coello

PARALELO: "A"

SEMESTRE: QUINTO

ACTIVIDAD: Actividad #1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO ABRIL - AGOSTO 2026



Actividades a desarrollar en la Unidad 1

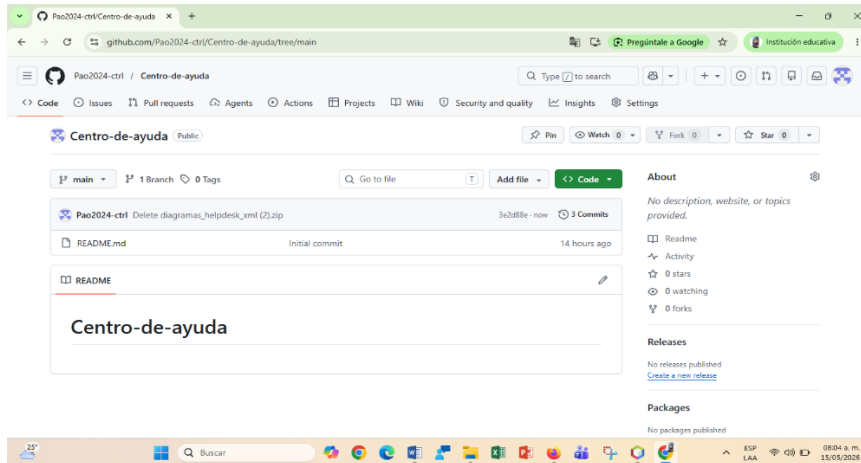
INTRODUCCION

En la actualidad, el soporte técnico eficiente es un factor clave para garantizar la continuidad operativa de cualquier organización. Los sistemas de gestión de incidentes en los help desk permiten registrar, clasificar, priorizar y resolver las solicitudes e incidencias reportadas por los usuarios de forma estructurada y trazable. Este estudio de caso se centra en analizar el funcionamiento de un sistema de gestión de incidentes dentro de un entorno de soporte técnico. Se busca identificar cómo se gestionan los reportes desde su creación hasta su cierre, qué actores intervienen en el proceso, y qué herramientas o buenas prácticas se aplican para reducir tiempos de respuesta y mejorar la satisfacción del usuario. A través del análisis del caso, se pretende comprender los desafíos comunes en la operación de un help desk, tales como la categorización incorrecta de incidentes, la sobrecarga de solicitudes, la falta de comunicación con el usuario final y las fallas en el seguimiento. A partir de esto, se podrán proponer mejoras que optimicen el flujo de trabajo y alineen el servicio con estándares como ITIL.

El objetivo final es evaluar la eficacia del sistema actual y determinar oportunidades de mejora que permitan una gestión más ágil, transparente y orientada al usuario.

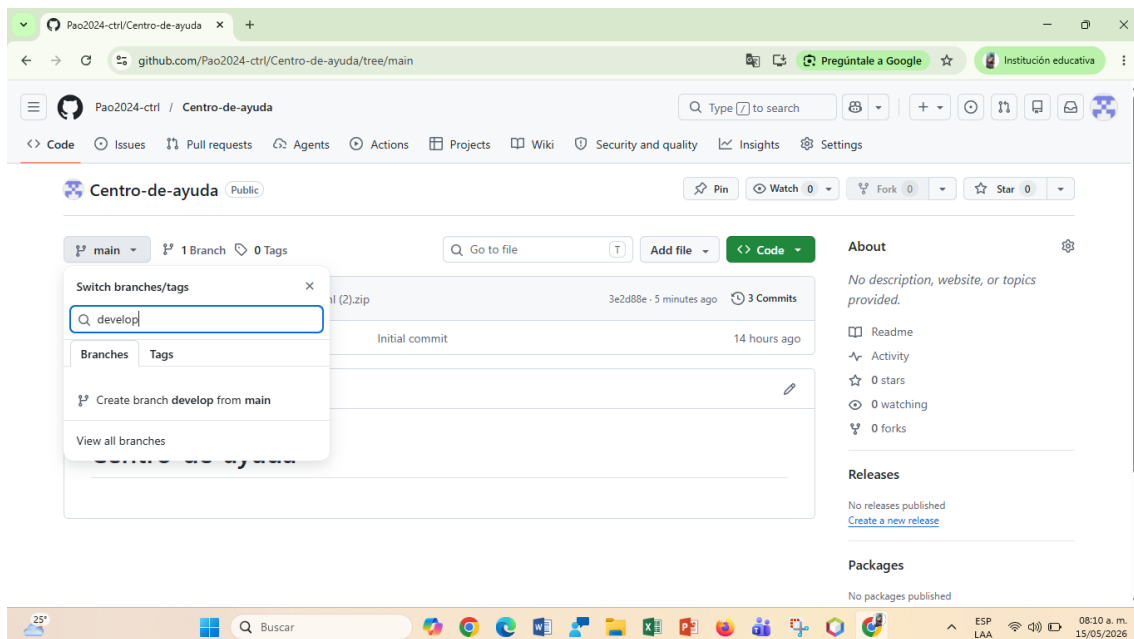
1. Creación del repositorio en GitHub

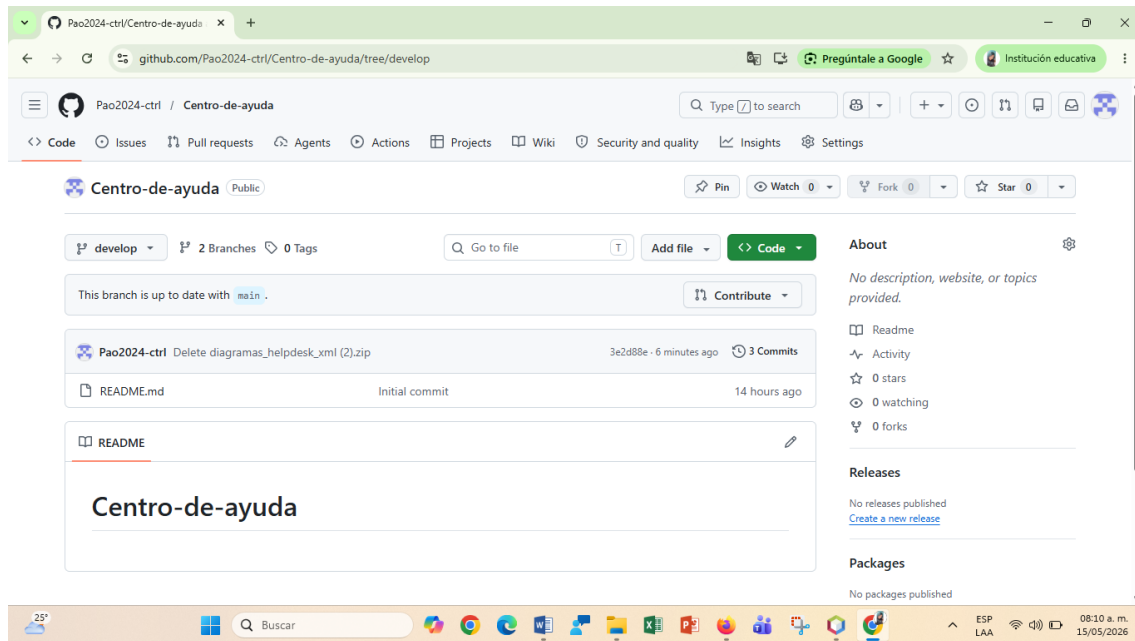
Como parte del desarrollo, se creó un repositorio público para gestionar el proyecto y respaldar todo el avance del análisis. En él se organizaron los documentos técnicos en carpetas dedicadas, donde se subieron archivos como analisis.md y los diagramas UML en formato PNG. Esto asegura que el trabajo quede versionado y accesible desde cualquier lugar, sirviendo como base para aplicar mejoras en fases posteriores. A partir de este caso se busca detectar fallas comunes como la mala categorización de incidentes, la falta de seguimiento y la comunicación deficiente con el usuario final. Con ese diagnóstico, se plantearán oportunidades de mejora que alineen el proceso con buenas prácticas como ITIL, buscando reducir tiempos de respuesta y aumentar la satisfacción del usuario.



2. Creación de la rama develop con Gitflow

Como parte del desarrollo, se creó una rama en el repositorio para separar el trabajo de análisis del código estable. En esa rama se integran los avances del análisis y los diagramas UML del sistema, usando carpetas dedicadas para organizar archivos como analisis.md y los diagramas en PNG. Esto facilita el control de versiones y la colaboración ordenada sin afectar la versión estable del proyecto. El objetivo es evaluar qué tan eficiente es el sistema actual y proponer mejoras basadas en buenas prácticas como ITIL, para reducir tiempos de respuesta y mejorar la experiencia del usuario final.

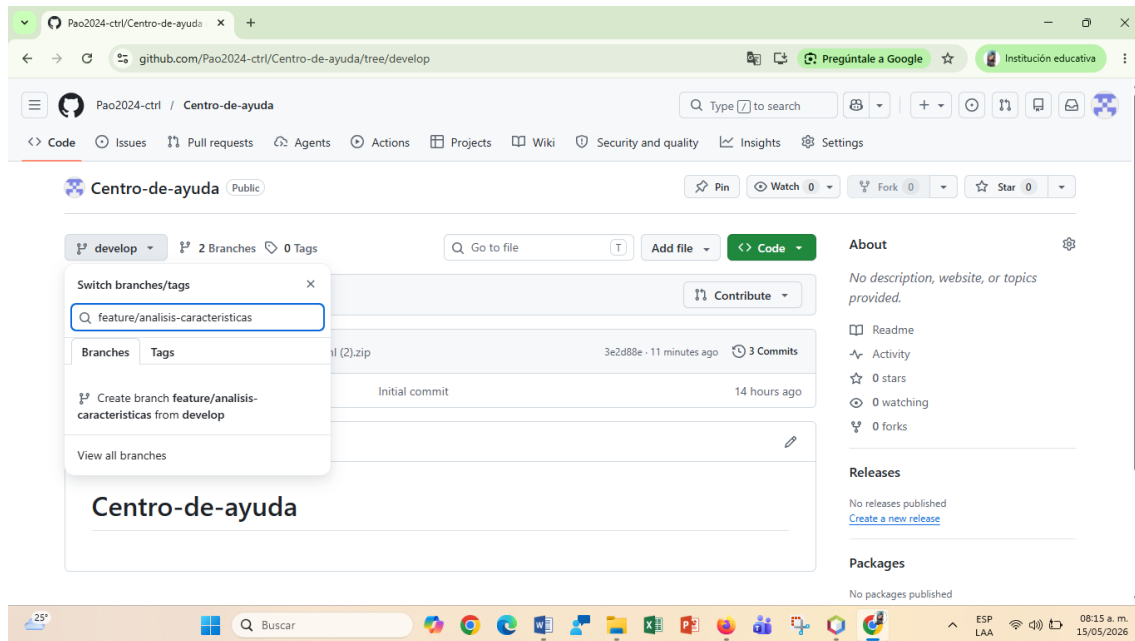




3. Creación de ramas feature con Gitflow

Para respaldar el trabajo, se utilizó un modelo de ramas tipo Git Flow: las ramas feature se crean desde develop para desarrollar funcionalidades específicas sin afectar las ramas principales. Cada rama lleva el nombre de la tarea, por ejemplo feature/analisis-caracteristicas, y una vez terminada se integra de vuelta a develop mediante un pull request. Esto mantiene el historial limpio y facilita la revisión del código, evitando trabajar directamente sobre develop o main para reducir errores.

El objetivo es evaluar la eficacia del sistema actual y proponer mejoras alineadas con buenas prácticas como ITIL, con el fin de reducir tiempos de respuesta y mejorar la satisfacción del usuario final.



4. Documentación del análisis en la rama feature

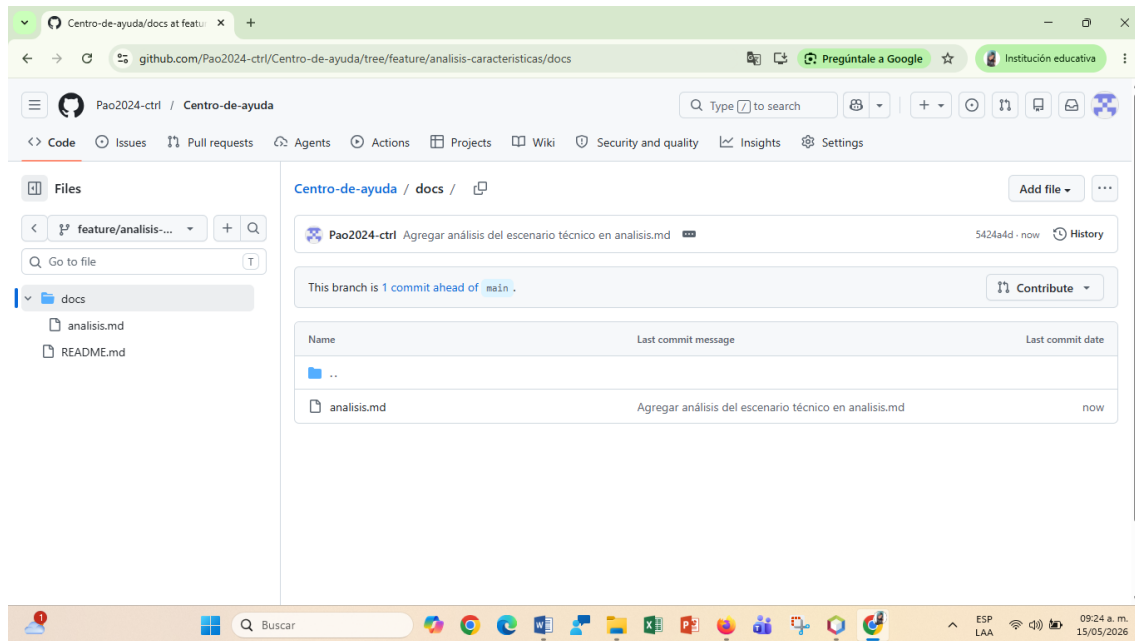
En la rama feature/analisis-caracteristicas se documentó el análisis del sistema Help Desk.

Se creó el archivo analisis.md con los requisitos funcionales y no funcionales identificados.

También se agregaron los diagramas UML de casos de uso, clases y secuencia en formato PNG.

Los archivos se organizaron dentro de la carpeta docs para mantener el proyecto ordenado.

Esta estructura permite que el equipo revise y valide el análisis antes de fusionar a develop. Trabajar en una rama feature evita modificar directamente la rama principal de desarrollo. Así se cumple con el flujo Gitflow y se deja un registro claro del trabajo realizado.



5. Creación del diagrama de casos de uso

En la rama `feature/analisis-caracteristicas` se hizo el análisis del sistema. Ahí se creó el archivo `analisis.md` donde se dejaron claros los requisitos funcionales y no funcionales. También se subieron los diagramas UML —casos de uso, clases y secuencia— en PNG. Todo quedó organizado dentro de la carpeta `Docs` para que el proyecto se mantenga ordenado. Esta forma de trabajar permite que el equipo revise y valide el análisis antes de fusionarlo a `develop`. Al usar una rama `feature` evitas tocar directamente la rama principal de desarrollo. Así se sigue el flujo de Git Flow y queda un registro claro de todo lo que se hizo.

1. Actores

Hay 3 actores representados con los muñecos:

- Usuario Final - Color verde
- Soporte técnico - Color naranja
- Administrador - Color rojo

2. Casos de uso

Los óvalos son las acciones que pueden hacer los actores:

- Crear Ticket - Aparece 7 veces en colores celeste, rosado y morado
- Consultar Ticket - Solo lo tiene el Usuario Final
- Autenticar Usuario - Óvalo naranja al que apuntan todos los casos de uso

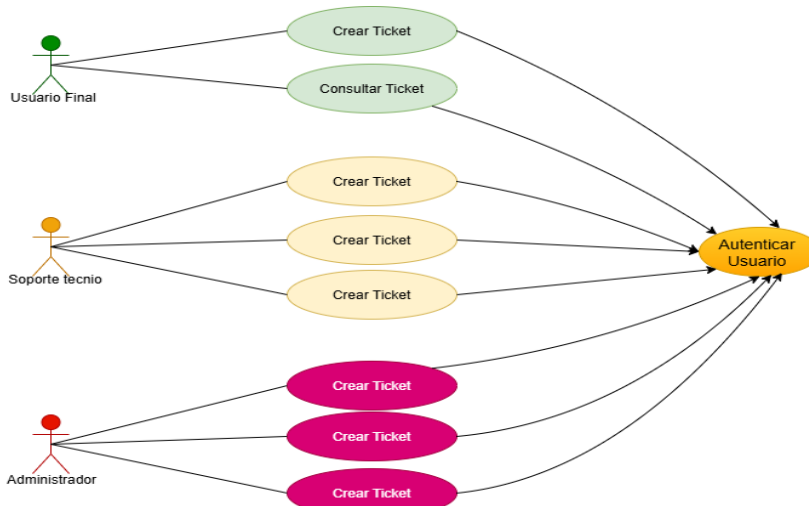
3. Relaciones

Las líneas conectan cada actor con sus casos de uso:

- Usuario Final: Se conecta a Crear Ticket y Consultar Ticket. Ambos apuntan a Autenticar Usuario.
- Soporte técnico: Se conecta a 3 óvalos de Crear Ticket que apuntan a Autenticar Usuario. Aquí parece haber un error, porque debería tener casos como Actualizar Ticket, Cerrar Ticket, Asignar Ticket.
- Administrador: Se conecta a 3 óvalos de Crear Ticket que apuntan a Autenticar Usuario. Debería tener Gestionar Usuarios, Ver Reportes, etc.

4. Observación

El diagrama tiene "Crear Ticket" repetido para Soporte y Administrador. Lo normal es que cada actor tenga casos de uso distintos según su rol. Si es para una entrega académica, súbelo así. Si lo puedes corregir, cambia los repetidos por acciones reales de cada rol.



7. DIAGRAMA DE CLASES

Un diagrama de clases es un diagrama UML que representa la estructura estática de un sistema orientado a objetos. Muestra las clases del sistema, sus atributos, métodos y las relaciones entre ellas. Cada clase se representa como un rectángulo dividido en tres partes: nombre, atributos y métodos. Las relaciones más comunes son asociación, herencia, agregación y composición, y se dibujan con líneas entre clases. Sirve para modelar la arquitectura lógica del sistema antes de programarlo. Permite visualizar cómo se organizan los datos y las responsabilidades dentro del software. Es útil para identificar duplicidad, dependencias y mejorar el diseño. Los desarrolladores lo usan como guía para



implementar el código. En un help desk, este diagrama modelaría entidades como Usuario, Ticket, Técnico y sus conexiones. Es la base para generar código y bases de datos a partir del diseño.

1. Clases y Atributos:

- USUARIO: id, nombre, contraseña, rol. Métodos: iniciarSesion(), cerrarSesion()
- TICKET: idTicket, titulo, descripcion, estado, fechaCreacion. Métodos: crearTicket(), cerrarTicket(), cambiarEstado()
- TECNICO: idTecnico, especialidad. Métodos: asignarTicket(), resolverTicket()
- ADMINISTRADOR: idAdmin, area. Métodos: gestionarUsuarios(), verReportes(), gestionarCategorias()
- CATEGORIA: idCategoria, nombre, descripcion. Métodos: crearCategoria(), editarCategoria()
- COMENTARIO: idComentario, mensaje, fecha. Método: crearComentario()

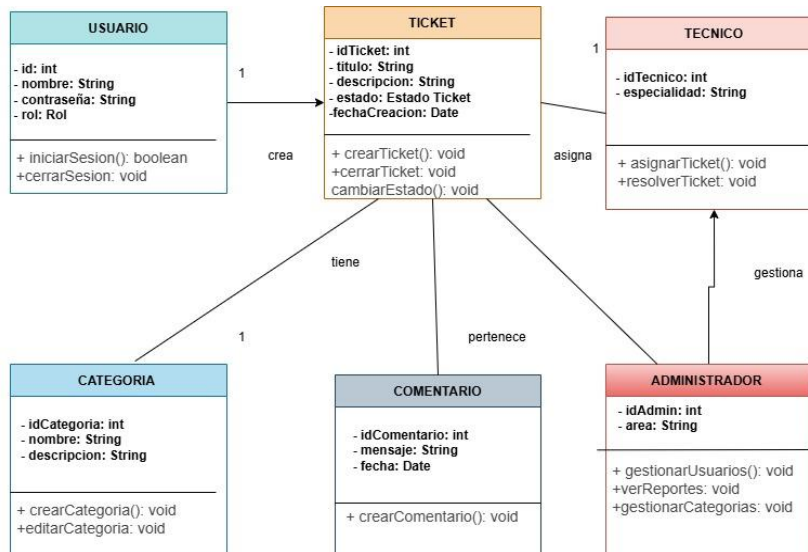
2. Relaciones:

- USUARIO crea TICKET: Un usuario puede crear múltiples tickets. Relación 1 a muchos.
- TICKET asigna TECNICO: Cada ticket es asignado a un técnico. Relación 1 a 1.
- TICKET tiene CATEGORIA: Cada ticket pertenece a una categoría. Relación 1 a 1.
- TICKET pertenece a COMENTARIO: Un ticket puede tener múltiples comentarios. Relación 1 a muchos.
- ADMINISTRADOR gestiona TECNICO: El administrador gestiona a los técnicos del sistema.

3. Interpretación:

El diagrama muestra que TICKET es la clase central del sistema. Los usuarios crean tickets, los técnicos los resuelven, y los administradores gestionan

usuarios, técnicos y categorías. La clase COMENTARIO permite el seguimiento y comunicación dentro de cada ticket.



8. DIAGRAMA DE SECUENCIA

Un diagrama de secuencia es un diagrama UML que representa la interacción entre los elementos de un sistema ordenada en el tiempo. Muestra cómo los actores y objetos se comunican entre sí para llevar a cabo un caso de uso específico. En él se colocan los actores y objetos en la parte superior como columnas y sus líneas de vida se extienden verticalmente hacia abajo. Las interacciones se representan con flechas horizontales llamadas mensajes que indican quién envía y quién recibe la información. El tiempo avanza de arriba hacia abajo, por lo que el orden de las flechas define la secuencia del proceso. Se utiliza para detallar el flujo interno de operaciones como un login, una compra o la creación de un ticket. Permite identificar qué objeto ejecuta cada acción y en qué momento lo hace. Es útil para validar que el diseño del sistema cumple con los requisitos funcionales. También facilita la comunicación entre analistas, desarrolladores y clientes al ser visual y fácil de seguir. En el contexto de un help desk sirve para modelar el proceso completo de gestión de un incidente.

Participantes:

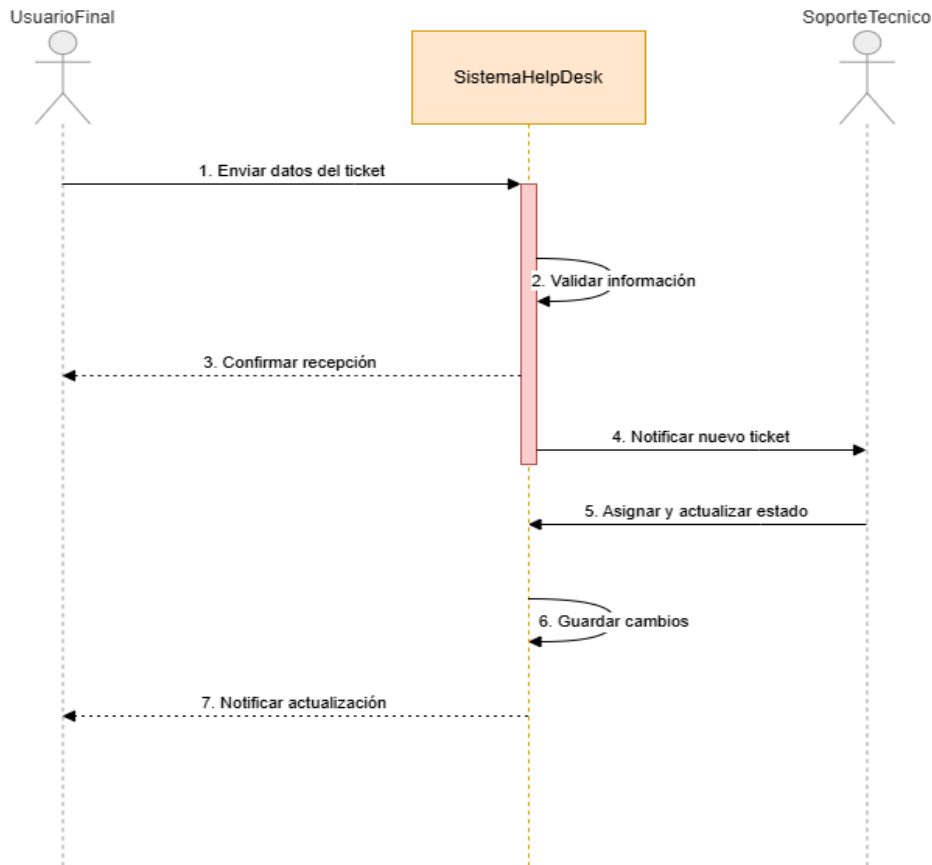
1. UsuarioFinal: Actor que reporta el incidente
2. SistemaHelpDesk: Sistema central que procesa la solicitud



3. SoporteTecnico: Actor que recibe y gestiona el ticket

Flujo de mensajes:

1. Enviar datos del ticket: El UsuarioFinal envía la información del incidente al SistemaHelpDesk
2. Validar información: El sistema verifica que los datos estén completos y correctos
3. Confirmar recepción: El sistema envía una confirmación al UsuarioFinal de que el ticket fue recibido
4. Notificar nuevo ticket: El sistema notifica a SoporteTecnico sobre el ticket creado
5. Asignar y actualizar estado: SoporteTecnico asigna el ticket y cambia su estado a "En proceso"
6. Guardar cambios: El sistema guarda la actualización en la base de datos
7. Notificar actualización: El sistema informa al UsuarioFinal sobre el cambio de estado del ticket



10. APLICACIÓN DE PATRONES DE DISEÑO

Patrón Observer

El patrón Observer define una relación de uno a muchos entre objetos para que cuando uno cambie de estado, todos los dependientes sean notificados automáticamente. Se usa para desacoplar el objeto que genera el evento, llamado sujeto, de los objetos que reaccionan a ese evento, llamados observadores. El sujeto mantiene una lista de observadores suscritos y les envía una notificación cuando ocurre un cambio. Cada observador implementa una interfaz común con un método de actualización. Así, el sujeto no necesita conocer detalles de cada observador, solo que implementan ese método. Esto permite agregar o quitar observadores en tiempo de ejecución sin modificar el sujeto. Es útil para eventos, actualizaciones de interfaz y sistemas de notificación. En un help desk serviría para notificar a técnicos cuando se crea un ticket nuevo. Reduce el acoplamiento y mejora la escalabilidad del sistema. El patrón sigue el principio de inversión de dependencias y favorece la flexibilidad.

Patrón Factory

El patrón Factory proporciona una interfaz para crear objetos sin exponer la lógica de instanciación al cliente. En lugar de usar `new` directamente, se llama a un método de una fábrica que devuelve la instancia adecuada según los parámetros recibidos. Esto oculta las clases concretas y permite cambiar la implementación sin modificar el código que usa el objeto. Facilita el mantenimiento porque toda la creación está centralizada en un solo lugar. Se usa cuando el sistema debe ser independiente de cómo se crean y representan los productos. Hay variantes como Factory Method y Abstract Factory según la complejidad. Permite aplicar el principio abierto/cerrado, ya que puedes agregar nuevos tipos sin tocar el código existente. Reduce el acoplamiento entre clases y mejora la legibilidad. En un help desk serviría para crear distintos tipos de tickets según su categoría. Es ideal cuando la creación de objetos es compleja o depende de condiciones.

Patrón Singleton

El patrón Singleton garantiza que una clase tenga una única instancia en toda la aplicación y proporciona un punto de acceso global a ella. Se logra ocultando el constructor y exponiendo un método estático que controla la creación de la instancia. La primera vez que se llama, crea el objeto y lo guarda; las siguientes veces devuelve el mismo objeto ya creado. Esto evita duplicar recursos costosos como conexiones a base de datos o configuraciones del sistema. El patrón asegura un estado compartido y consistente en todo el programa. Es útil cuando solo debe existir un coordinador central, como un gestor de logs o de configuración. Hay que cuidar su uso en entornos concurrentes para evitar crear múltiples instancias. En un help desk serviría para un gestor de sesiones o un logger único. Reduce el consumo de memoria al evitar objetos redundantes. Sin embargo, su uso excesivo puede dificultar las pruebas y violar el principio de responsabilidad única. Importancia de estos patrones utilizados

La aplicación de Singleton, Observer y Factory mejora la organización, escalabilidad y mantenibilidad del sistema Help Desk. Singleton optimiza recursos al centralizar servicios críticos como la conexión a BD. Observer permite un sistema de notificaciones desacoplado y extensible. Factory facilita



la creación de objetos complejos y la incorporación de nuevos tipos de ticket. En conjunto, estos patrones siguen principios SOLID y buenas prácticas de ingeniería de software, asegurando un sistema robusto y fácil de evolucionar.

CONCLUSION

Un sistema de gestión de incidentes centraliza y ordena todas las solicitudes de soporte en un help desk, evitando que se pierdan casos y permitiendo priorizarlos según su impacto y urgencia. Esto mejora directamente la respuesta ante fallas críticas y reduce los tiempos de resolución, especialmente cuando se automatizan notificaciones y escalamientos para disminuir la carga manual del equipo.

Además, el registro histórico que generan estos sistemas facilita analizar las causas raíz y prevenir problemas recurrentes. Mejora la comunicación con los usuarios al darles seguimiento en tiempo real, y aumenta la trazabilidad y transparencia del proceso, lo que genera mayor confianza en el servicio.

Por último, estos sistemas permiten medir el desempeño mediante indicadores y reportes, integrarse con correo, chat y bases de conocimiento, y elevar la calidad del servicio. Pero para que funcionen bien se necesita capacitar al personal y definir claramente los procesos y roles. En resumen, adoptar un sistema así eleva tanto la satisfacción del usuario como la eficiencia operativa del help desk.

EVIDENCIAS

```
package patronesejemplos;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class GestorPedidos {
```

```
    private List<IObservador> observadores = new ArrayList<>();
```



```
public void agregarObservador(IObservador observador) {
    observadores.add(observador);
}

public void notificarObservadores(String mensaje) {
    for (IObservador obs : observadores) {
        obs.actualizar(mensaje);
    }
}

public void crearNuevoPedido(String tipo) {
    System.out.println("\n=== SOLICITUD DE NUEVO PEDIDO ===");

    Pedido nuevoPedido = PedidoFactory.crearPedido(tipo);

    if (nuevoPedido != null) {
        nuevoPedido.mostrarDetalles();
        notificarObservadores("Se ha creado un nuevo pedido de tipo " + tipo);
    } else {
        System.out.println("Error: Tipo de pedido no soportado.");
    }
}

public static void main(String[] args) {
    GestorPedidos gestor = new GestorPedidos();

    Cliente cliente1 = new Cliente("Juan Perez");
    Cliente cliente2 = new Cliente("Maria Gomez");

    gestor.agregarObservador(cliente1);
    gestor.agregarObservador(cliente2);
}
```



```
gestor.crearNuevoPedido("Digital");
```

```
}
```

```
}
```

The image shows a Google Drive folder named 'src' containing several Java files. Below this, a NetBeans IDE window displays the code for 'GestorPedidos.java'. The code implements a simple observer pattern for a ticket management system. The output window shows the result of running the program, indicating that a digital ticket was successfully created and notified to the client.

Google Drive Folder: src

Nombre	Propietario	Fecha de modificación	Tamaño de archivo	Ordenar
TicketHardware.java	yo	13 may '20	190 bytes	
TicketFactory.java	yo	13 may '20	216 bytes	
Ticket.java	yo	13 may '20	57 bytes	
Tecnico.java	yo	13 may '20	308 bytes	
IObservador.java	yo	13 may '20	71 bytes	
GestorTickets.java	yo	13 may '20	2 KB	

NetBeans IDE: GestorPedidos.java

```
1 package patronesejemplos;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class GestorPedidos {
7     private List<IObservador> observadores = new ArrayList<>();
8
9     public void agregarObservador(IObservador observador) {
10         observadores.add(observador);
11     }
12
13     public void notificarObservadores(String mensaje) {
14         for (IObservador obs : observadores) {
15             obs.actualizar(mensaje);
16         }
17     }
18
19     public void crearNuevoPedido(String tipo) {
20         System.out.println("\n== SOLICITUD DE NUEVO PEDIDO ==");
21     }
22 }
```

Output:

```
NetBeansProjects - C:\Users\Usuario\Documents\NetBeansProjects \ Patronesejemplos (run) X
run:
== SOLICITUD DE NUEVO PEDIDO ==
Pedido: Producto digital registrado y listo para descarga.
Notificando al Cliente (Juan Perez): Se ha creado un nuevo pedido de tipo Digital
Notificando al Cliente (Maria Gomez): Se ha creado un nuevo pedido de tipo Digital
BUILD SUCCESSFUL (total time: 0 seconds)
```



Link de github

<https://github.com/Pao2024-ctrl/Centro-de-ayuda>

Link del drive

<https://drive.google.com/drive/folders/1H2vXBrkBzGU81WEWCqmlIbT-PKPKV5ik?usp=sharing>

BIBLIOGRAFIA

https://www.google.com/search?gs_ssp=eJzj4tLP1TcoqiwoyE1XYDRgdGDwYkvPLMk_oTQIAWC4HFA&q=github&og=GI&gs_lcrp=EgZjaHJvbWUqEwgBEC4YgwEYxwEYsQMY0QMYgAQyBggAEEUYOTITCAEQLhiDARjHARixAxiRAXiABDINCAIQLhiDARixAxiABDINCAMQABiDARixAxiABDIGCAQQRRg9MgYIBRBFGDwyBggGEEUYPDIGCAcQRRg80gEIMzg5NGowajeoAgiwAgHxBS1OxR28LVBc&sourceid=chrome&ie=UTF-8

https://www.google.com/search?q=draw+io&og=DRA&gs_lcrp=EgZjaHJvbWUqFAgAEUYOxhDGIMBGLEDGIAEGIoFMhQIABBFGDsYQxiDARixAxiABBiKBTISCAEQABhDGIMBGLEDGIAEGIoFMgwIAhAAGEMYgAQYigUyEggDEC4YQxiDARixAxiABBiKBTIGCAQQRRg5MgYIBRBFGDwyBggGEEUYPDIGCAcQRRg80gEIMzQ4OGowajmoAgawAgHxBV_6gkkNwtm8&sourceid=chrome&ie=UTF-8